Meeting 6 SP23

CPUs vs. GPUs (and a little hint of CUDA)

Aydan Pirani

University of Illinois Urbana-Champaign

March 22, 2023

Outline

High-Level CPU Overview

The More (cores), the Merrier!

Moore's Law: A Not-So-Little Problem

GPUs: Doing What CPUs Never Could

Trivia Night

Section 1

High-Level CPU Overview

A Few Questions

- What does CPU stand for?

A Few Questions

- What does CPU stand for? **Central Processing Unit.**
- What is a CPU?

A Few Questions

- What does CPU stand for? **Central Processing Unit.**
- What is a CPU? **The core component, that allows computers to process things.**
- What does a CPU do?

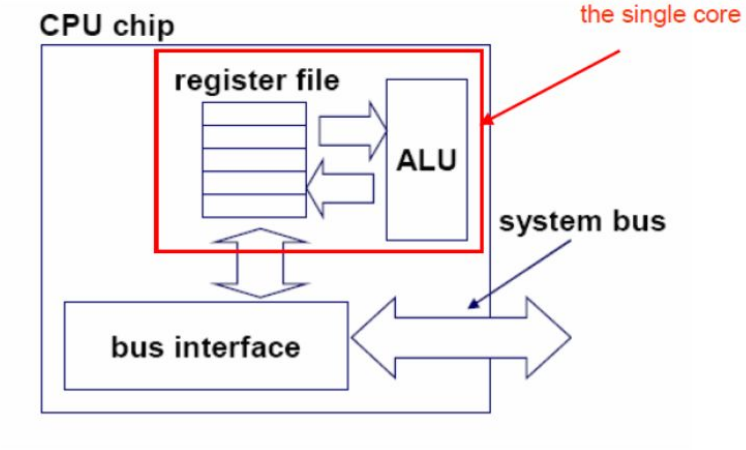
A Few Questions

- What does CPU stand for? **Central Processing Unit.**
- What is a CPU? **The core component, that allows computers to process things.**
- What does a CPU do? **Read/process instructions, perform calculations, and read/write to memory.**

A Few Questions

- What does CPU stand for? **Central Processing Unit.**
- What is a CPU? **The core component, that allows computers to process things.**
- What does a CPU do? **Read/process instructions, perform calculations, and read/write to memory.**

Single-core CPU chip



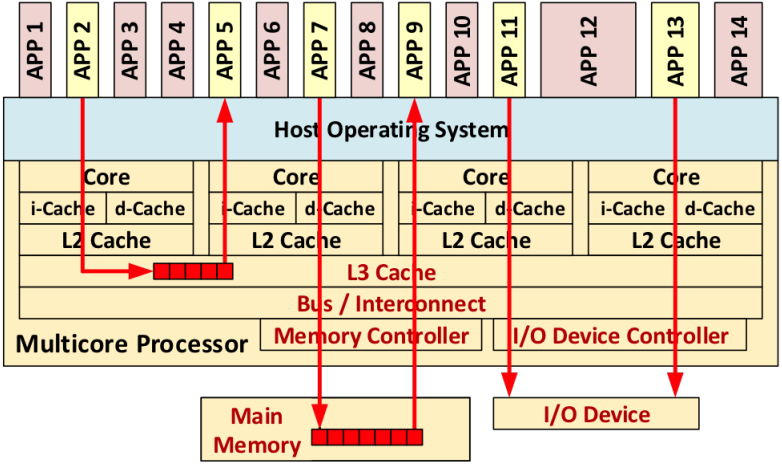
Multithreading

- One core can generally only run one thread at once.
- (Technologies do exist to mitigate this, such as hyperthreading).
- To prevent wastage of power, we put more cores onto machines.

Multi-Core Machines

- Cores are placed together onto chips to form processors.
- Processors share a bus to access memory and IO operations (and sometimes caches).
- There are multiple tiers of caches: L1, L2, L3.

Multi-Core CPU Design



Section 3

Moore's Law: A Not-So-Little Problem

Two Types of Problems

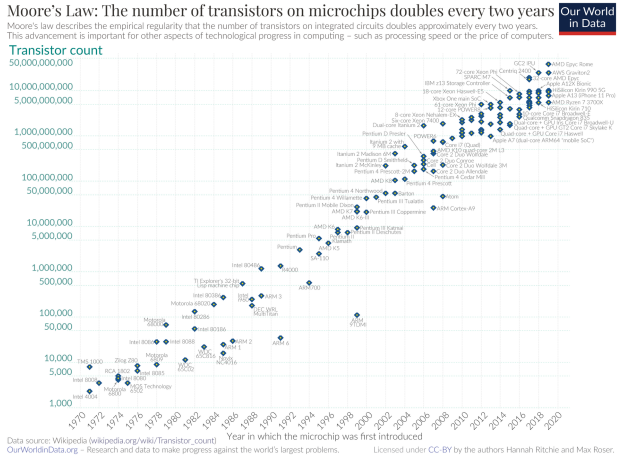
- Computationally-Intensive: A SINGLE PROBLEM takes up a lot of time to compute.
- Data-Intensive: The problem is split into MANY SUBPROBLEMS, each of which are easy to solve.
- Is there a problem here?

Two Types of Problems

- Computationally-Intensive: A SINGLE PROBLEM takes up a lot of time to compute.
- Data-Intensive: The problem is split into MANY SUBPROBLEMS, each of which are easy to solve.
- **Note that solving data-intensive problems on a CPU is a waste of computational power.**

Moore's Law

- The number of transistors on a chip will double approximately every two years.



[ht!]

Implications of Moore's Law

- Note that we're reaching a very important point!
- Hm, this seems to look like it's tapering off...
- Why does this happen?

Implications of Moore's Law

- Note that we're reaching a very important point!
- Hm, this seems to look like it's tapering off...
- Why does this happen? **BECAUSE POWER!!!**

Here's the Problem

- As we make more and more powerful chips, the amount of power increases exponentially.
- The energy needed to cool the chips is too much, and providing this much energy consistently is unfeasible.
- Note that this is literally a hardware problem, not a software problem! (We have to work around it, as opposed to fixing it somehow.)

One Potential Solution

- If we have a limited amount of cores and threads, what if we have a LOT of less powerful cores?
- (This is analogous to scaling out, as opposed to scaling up).

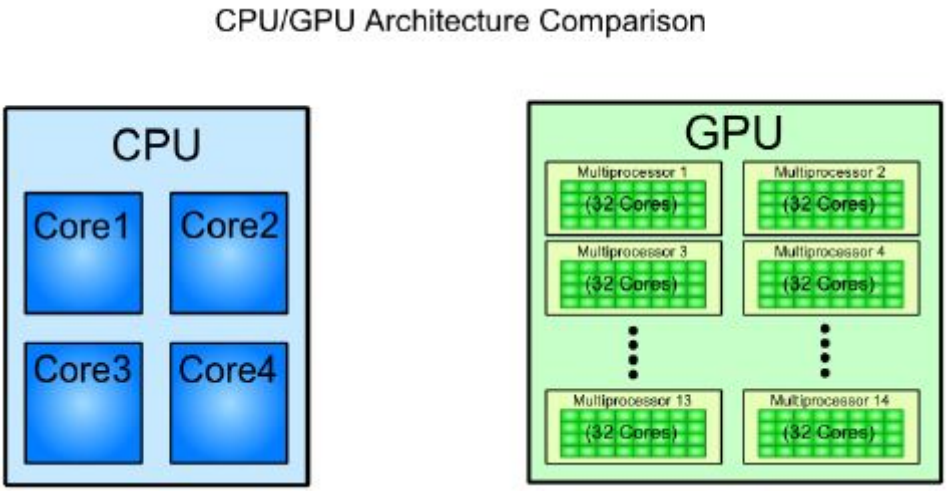
Section 4

GPUs: Doing What CPUs Never Could

What is a GPU?

- Instead of having a small number of powerful cores, GPUs have multiprocessors, which consist of many cores.
- Cores can run instructions simultaneously,
- Each core is less powerful!

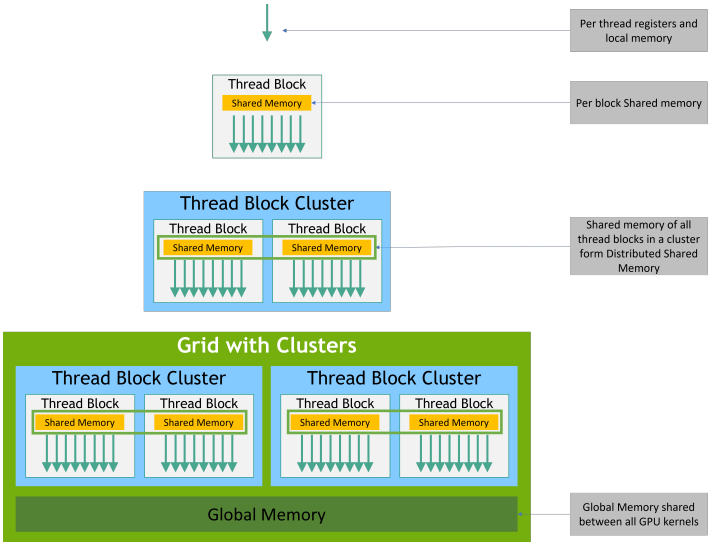
GPU Architecture



How Do Instructions Run?

- **Threads:** Computes operations/runs code on a core.
- **Warp:** Set of threads that runs together at the same time.
- **Thread Block:** Set of threads that runs together (but not simultaneously). Normally capped at 1024. Shares some memory.
- **Thread Grid:** Set of thread blocks that executes together, on multiple multiprocessors.

Memory on a GPU



Example CPU Code

```
void vectorAdd(
    const float *A, const float *B, float *C,
    int numElements) {

    for (int i = 0; i < numElements; i++) {
        C[i] = A[i] + B[i] + 0.0f;
  }
}
```

Example GPU Kernel Code

```
__global__ void vectorAdd(
    const float *A, const float *B, float *C,
    int numElements) {
  int i = blockDim.x * blockIdx.x + threadIdx.x;

  if (i < numElements) {
    C[i] = A[i] + B[i] + 0.0f;
  }
}
```

Example GPU Kernel Code

```
int main()
{
    cudaMalloc(A, ...);
    cudaMalloc(C, ...);
    cudaMalloc(B, ...);
    ...
    cudaMemcpy(A, ...);
    cudaMemcpy(B, ...);
    cudaMemcpy(C, ...);
    VecAdd<<<1, N>>>(A, B, C);
    __syncthreads();
    cudaFree(A); cudaFree(B); cudaFree(C);
    ...
}
```

Section 5

Trivia Night

Trivia Night

- For each of the following, guess if the tasks are better in the CPU or the GPU!

Trivia Night

- Dot Product!

Trivia Night

- Server-side socket code!

Trivia Night

- Graphics Processing!

Trivia Night

- Matrix Multiplication!

Trivia Night

- Prefix Sums!

The end, thanks for listening! Any questions?